Mutchumari Chandrasekhar Reddy AP25110010976

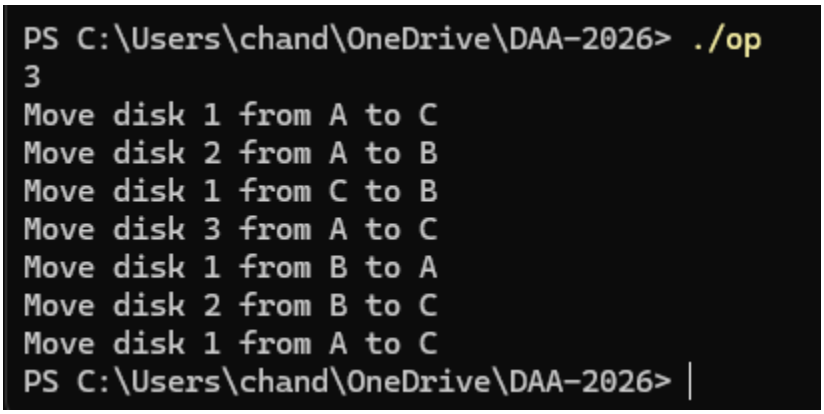
1.Converting recursive programs to non-recursive programs. Towers of Hanoi Problem example.

a. Implement using Recursion:

Program:

```
#include<stdio.h>

void hanoi(int n,char a,char b,char c){

if(n==1){

printf("Move disk 1 from %c to %c\n",a,c);

return;

}

hanoi(n-1,a,c,b);

printf("Move disk %d from %c to %c\n",n,a,c);

hanoi(n-1,b,a,c);

}

int main(){

int n;

scanf("%d",&n);

hanoi(n,'A','B','C');

return 0;

}
```

Terminal process:



b. Implement without Recursion:

```
#include<stdio.h>

#include<math.h>

void move(int n,char a,char c){

printf("Move disk %d from %c to %c\n",n,a,c);

}

void hanoi(int n,char a,char b,char c){

int i;

char x,y,z,temp;

x=a;

y=b;

z=c;

if(n%2==0){

temp=y;

y=z;

z=temp;

}
```

```
for(i=1;i<=pow(2,n)-1;i++){
if(i%3==1){
move((int)log2(i)+1,x,z);
}else if(i%3==2){
move((int)log2(i)+1,x,y);
}else{
move((int)log2(i)+1,y,z);
}
}
}
int main(){
int n;
scanf("%d",&n);
hanoi(n,'A','B','C');
return 0;
}
```

Terminal process:

```
PS C:\Users\chand\OneDrive\DAA-2026> gcc tower_nonrec.c -o op
PS C:\Users\chand\OneDrive\DAA-2026> ./op
3
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 2 from B to C
Move disk 3 from A to C
Move disk 3 from A to B
Move disk 3 from B to C
Move disk 3 from A to C
PS C:\Users\chand\OneDrive\DAA-2026>
```

2.Write a program to implement Stack operations using Linked List.

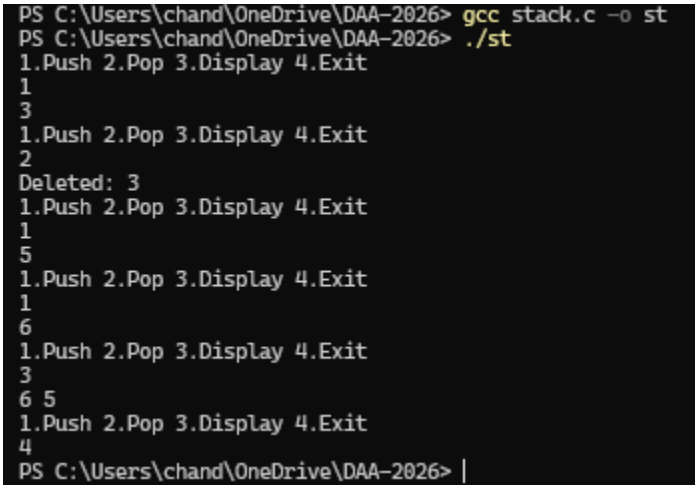
Program:

```
#include<stdio.h>
 #include<stdlib.h>
struct node{
int data; struct node*next;
};
 struct node*top=NULL;
void push(int x){
 struct node*newnode=(struct node*)malloc(sizeof(struct node));
newnode->data=x;
newnode->next=top;
 top=newnode;
}
void pop(){
struct node*temp;
 if(top==NULL){
printf("Stack Underflow\n");
return;
 }
 temp=top;
 printf("Deleted: %d\n",top->data);
top=top->next; free(temp);
 }
 void display(){
```

```
 struct node*temp=top;
if(top==NULL){
printf("Stack is empty\n");
 return;
 }
 while(temp!=NULL){
 printf("%d ",temp->data);
temp=temp->next;
}
printf("\n");
 }
 int main(){
int ch,x;
 while(1){
 printf("1.Push 2.Pop 3.Display 4.Exit\n");
 scanf("%d",&ch);
switch(ch){
case 1: scanf("%d",&x);
 push(x);
 break;
 case 2:
 pop();
break;
```

```
 case 3:
display();
 break;
 case 4:
return 0;
 }
 }
 }
```

Terminal process:



3.Write a program to implement Queue operations using Linked List.

Program:

```
#include<stdio.h>
 #include<stdlib.h>
 struct node{
```

```
int data;
 struct node*next;
 };
 struct node*front=NULL;
struct node*rear=NULL;
void enqueue(int x){
 struct node*newnode=(struct node*)malloc(sizeof(struct node));
 newnode->data=x;
newnode->next=NULL;
if(rear==NULL){
 front=rear=newnode;
 }else{
rear->next=newnode;
rear=newnode;
}
}
void dequeue(){
struct node*temp;
 if(front==NULL){
printf("Queue Underflow\n");
 return;
 }
 temp=front;
```

```
 printf("Deleted: %d\n",front->data);
 front=front->next;
if(front==NULL)
rear=NULL;
 free(temp);
 }
 void display(){
 struct node*temp=front;
if(front==NULL){
 printf("Queue is empty\n");
 return;
}
while(temp!=NULL){
 printf("%d ",temp->data);
temp=temp->next;
 }
 printf("\n");
 }
 int main(){
 int ch,x;
 while(1){
printf("1.Enqueue 2.Dequeue 3.Display 4.Exit\n");
 scanf("%d",&ch);
```

```
switch(ch)
{
case 1:
enqueue(x);
 break;
 case 2:
 dequeue();
 break;
 case 3:
 display();
break;
 case 4:
 return 0;
}
 }
 }
```

Terminal process:

```
PS C:\Users\chand\OneDrive\DAA-2026> gcc queue.c -o queu
PS C:\Users\chand\OneDrive\DAA-2026> ./queue
1.Enqueue 2.Dequeue 3.Display 4.Exit
1
3
1.Enqueue 2.Dequeue 3.Display 4.Exit
1
3
1.Enqueue 2.Dequeue 3.Display 4.Exit
1
8
1.Enqueue 2.Dequeue 3.Display 4.Exit
3
3 3 8
1.Enqueue 2.Dequeue 3.Display 4.Exit
2
Deleted: 3
1.Enqueue 2.Dequeue 3.Display 4.Exit
2
Deleted: 3
1.Enqueue 2.Dequeue 3.Display 4.Exit
2
Deleted: 8
1.Enqueue 2.Dequeue 3.Display 4.Exit
2
Queue Underflow
1.Enqueue 2.Dequeue 3.Display 4.Exit
4
PS C:\Users\chand\OneDrive\DAA-2026>
```